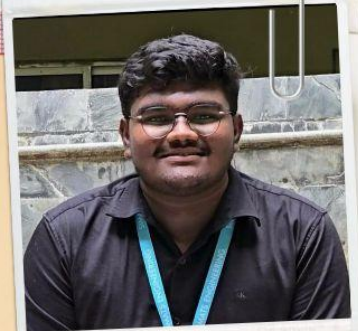


Lab TOPIC 1

08 OCTOBER 2026



1. First Palindromic String

CODE:

```
def firstPalindrome(words):
```

```
    for word in words:
```

```
        if word == word[::-1]:
```

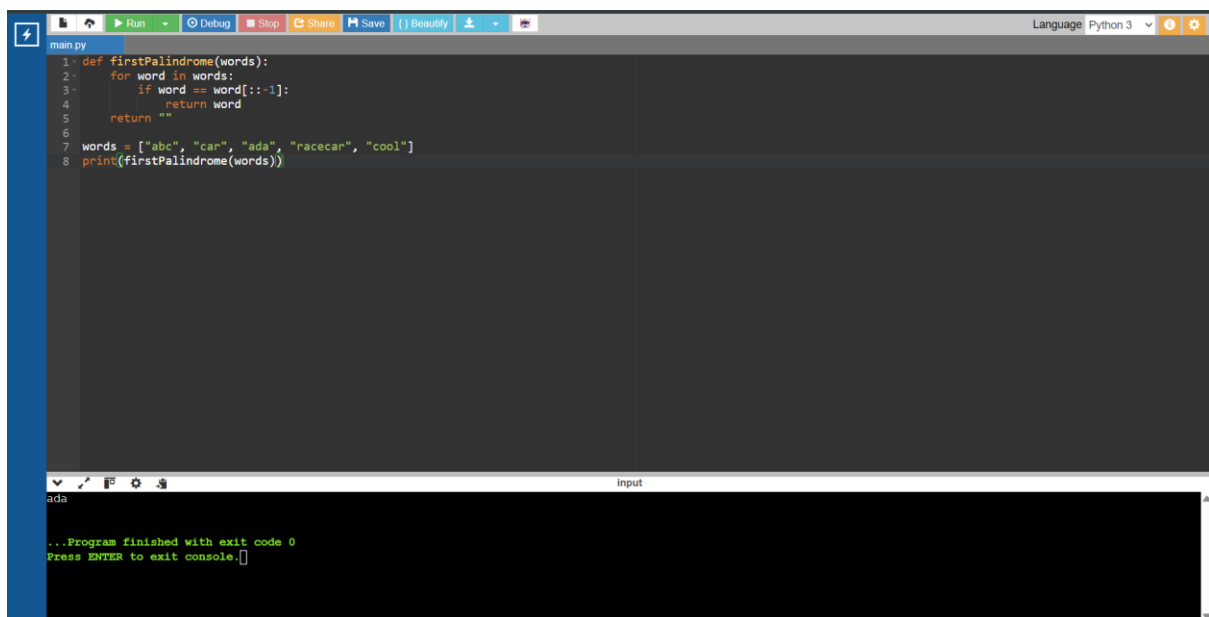
```
            return word
```

```
    return ""
```

```
words = ["abc", "car", "ada", "racecar", "cool"]
```

```
print(firstPalindrome(words))
```

OUTPUT:

A screenshot of a Python IDE window. The top toolbar includes buttons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The editor shows the following code:

```
1 def firstPalindrome(words):  
2     for word in words:  
3         if word == word[::-1]:  
4             return word  
5     return ""  
6  
7 words = ["abc", "car", "ada", "racecar", "cool"]  
8 print(firstPalindrome(words))
```

The output console at the bottom shows the word "ada" and a message: "...Program finished with exit code 0. Press ENTER to exit console."

2. Intersection Values

CODE:

```
def findIntersectionValues(nums1, nums2):
```

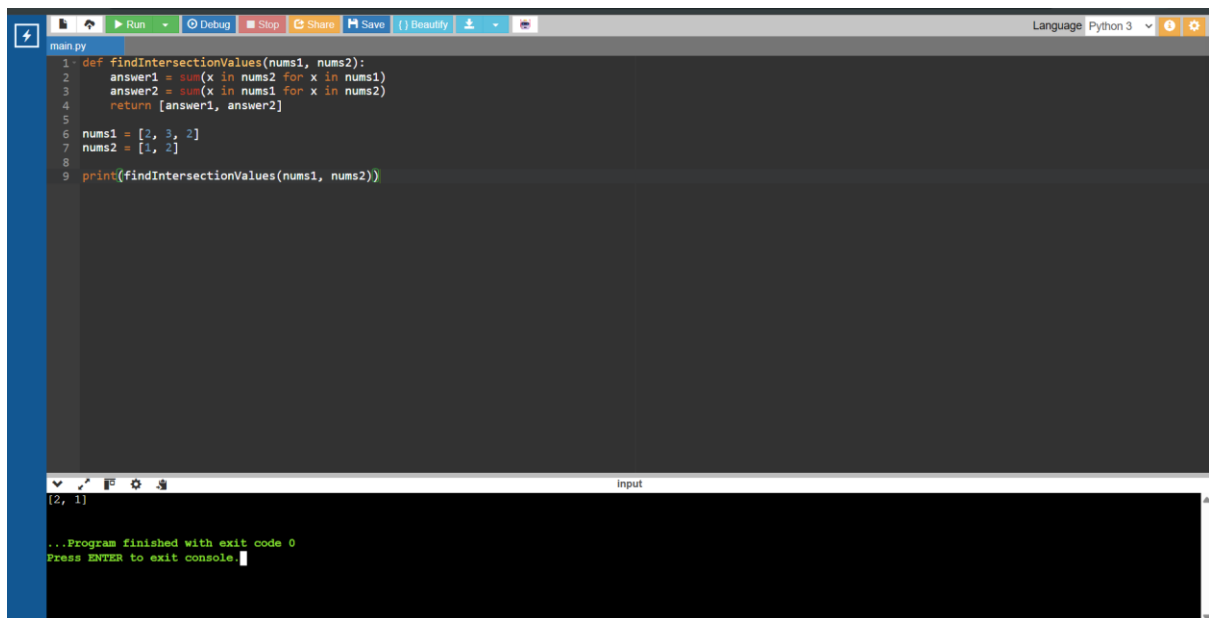
```
    a = sum(x in nums2 for x in nums1)
```

```
    b = sum(x in nums1 for x in nums2)
```

```
    return [a, b]
```

```
print(findIntersectionValues([2, 3, 2], [1, 2]))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py'. The code is as follows:

```
1 def findIntersectionValues(nums1, nums2):
2     answer1 = sum(x in nums2 for x in nums1)
3     answer2 = sum(x in nums1 for x in nums2)
4     return [answer1, answer2]
5
6 nums1 = [2, 3, 2]
7 nums2 = [1, 2]
8
9 print(findIntersectionValues(nums1, nums2))
```

The output console at the bottom shows the result of the execution:

```
[2, 1]
```

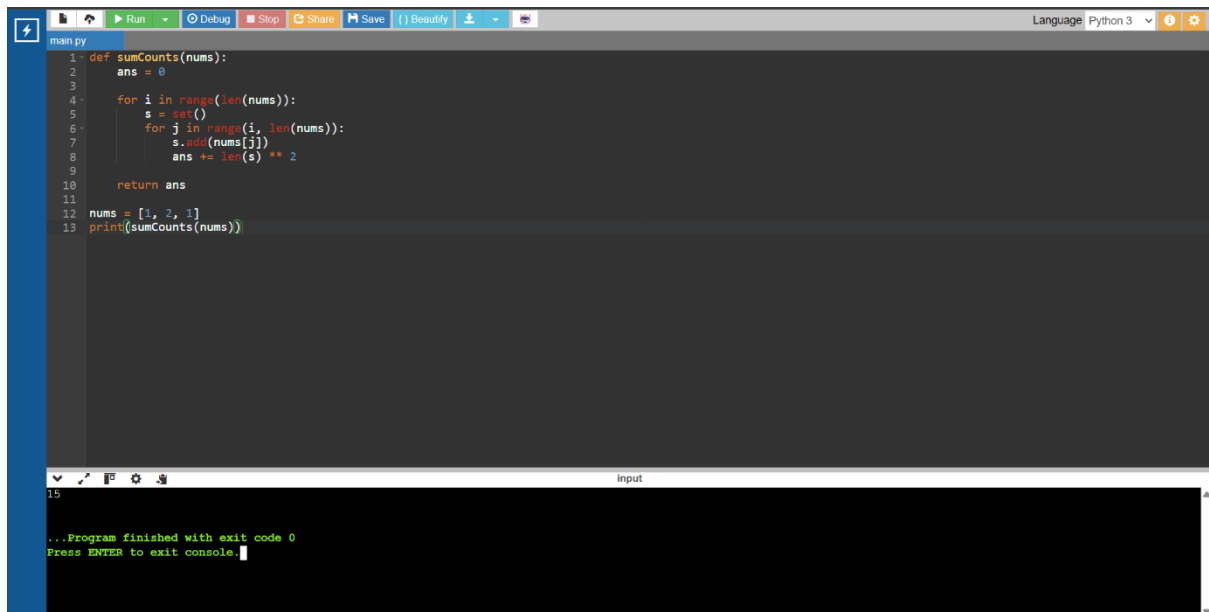
Below the output, the console displays the message: "...Program finished with exit code 0. Press ENTER to exit console."

3. Sum of Squares of Distinct Counts

CODE:

```
def sumCounts(nums):  
    ans = 0  
  
    for i in range(len(nums)):  
        s = set()  
        for j in range(i, len(nums)):  
            s.add(nums[j])  
            ans += len(s) ** 2  
  
    return ans  
  
print(sumCounts([1, 2, 1]))
```

OUTPUT:

A screenshot of a Python IDE window. The top bar shows 'main.py' and 'Language: Python 3'. The editor area contains the following code:

```
1 def sumCounts(nums):  
2     ans = 0  
3  
4     for i in range(len(nums)):  
5         s = set()  
6         for j in range(i, len(nums)):  
7             s.add(nums[j])  
8             ans += len(s) ** 2  
9  
10        return ans  
11  
12    nums = [1, 2, 1]  
13    print(sumCounts(nums))
```

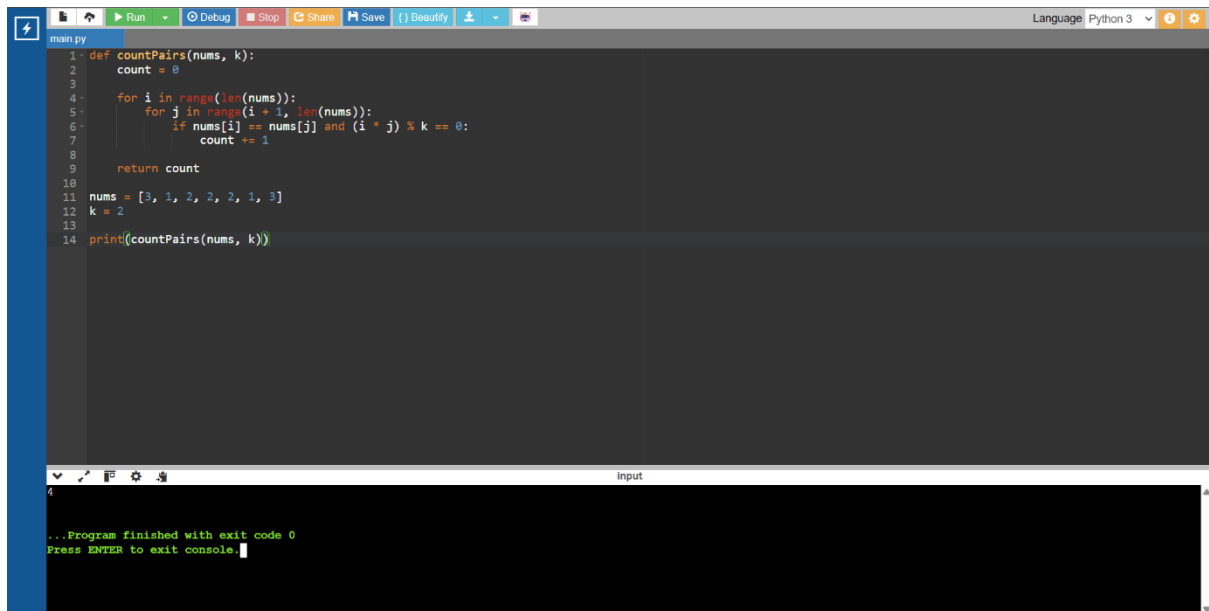
The bottom panel shows the output: '15' followed by '...Program finished with exit code 0' and 'Press ENTER to exit console.'.

4. Count Valid Pairs

CODE:

```
def countPairs(nums, k):  
    count = 0  
  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] == nums[j] and (i * j) % k == 0:  
                count += 1  
  
    return count  
  
print(countPairs([3, 1, 2, 2, 2, 1, 3], 2))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py'. The code in the editor is as follows:

```
1 def countPairs(nums, k):  
2     count = 0  
3  
4     for i in range(len(nums)):  
5         for j in range(i + 1, len(nums)):  
6             if nums[i] == nums[j] and (i * j) % k == 0:  
7                 count += 1  
8  
9     return count  
10  
11 nums = [3, 1, 2, 2, 2, 1, 3]  
12 k = 2  
13  
14 print(countPairs(nums, k))
```

The IDE interface includes a top toolbar with buttons for Run, Debug, Stop, Share, Save, and Beautify. The bottom panel shows the console output:

```
...Program finished with exit code 0  
Press ENTER to exit console.
```

5. Find Maximum Element

CODE:

```
def findMax(nums):
```

```
    maximum = nums[0]
```

```
    for x in nums:
```

```
        if x > maximum:
```

```
            maximum = x
```

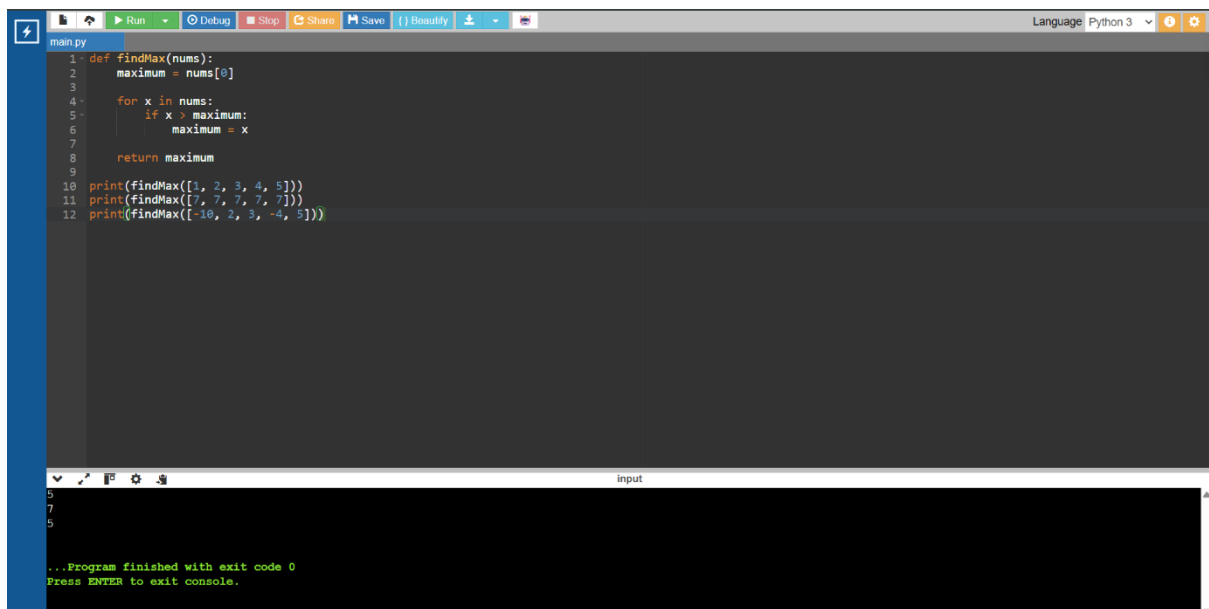
```
    return maximum
```

```
print(findMax([1, 2, 3, 4, 5]))
```

```
print(findMax([7, 7, 7, 7, 7]))
```

```
print(findMax([-10, 2, 3, -4, 5]))
```

OUTPUT:



The screenshot shows a Python IDE with a dark theme. The top toolbar includes buttons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The code editor displays the following Python code:

```
1 def findMax(nums):
2     maximum = nums[0]
3
4     for x in nums:
5         if x > maximum:
6             maximum = x
7
8     return maximum
9
10 print(findMax([1, 2, 3, 4, 5]))
11 print(findMax([7, 7, 7, 7, 7]))
12 print(findMax([-10, 2, 3, -4, 5]))
```

Below the code editor is an input field and a console window. The console shows the output of the program:

```
5
7
5
...Program finished with exit code 0
Press ENTER to exit console.
```

6. Find Max

CODE:

```
def findMax(nums):
```

```
    if not nums:
```

```
        return None
```

```
    nums.sort()
```

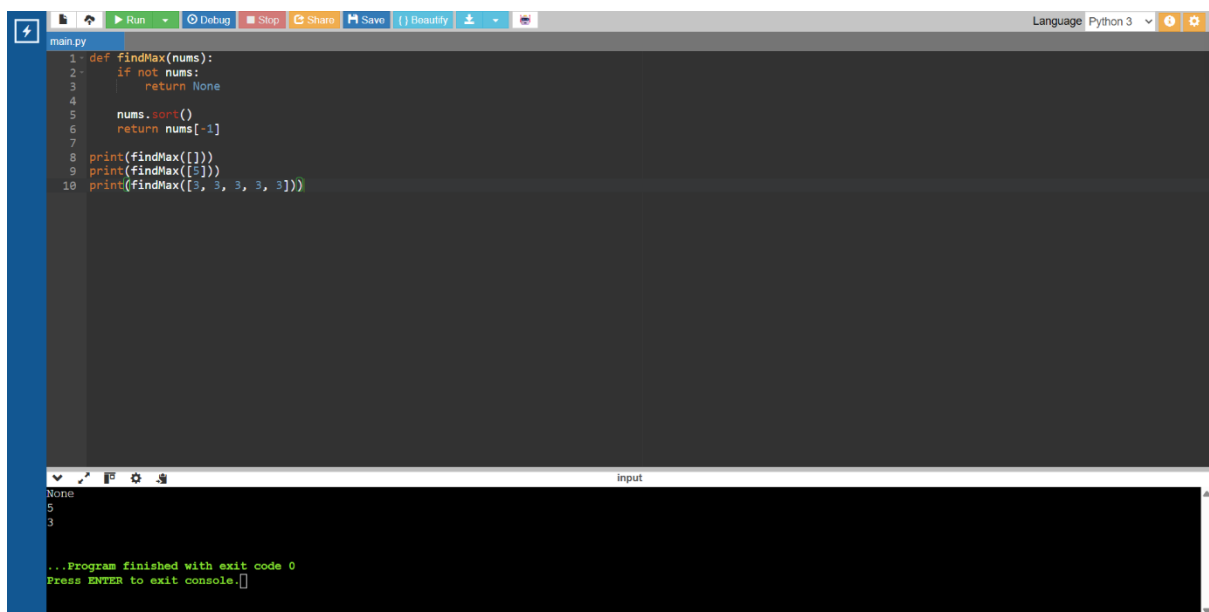
```
    return nums[-1]
```

```
print(findMax([]))
```

```
print(findMax([5]))
```

```
print(findMax([3, 3, 3, 3, 3]))
```

OUTPUT:

A screenshot of a Python IDE window titled 'main.py'. The code editor shows the following Python code:

```
1 def findMax(nums):
2     if not nums:
3         return None
4
5     nums.sort()
6     return nums[-1]
7
8 print(findMax([]))
9 print(findMax([5]))
10 print(findMax([3, 3, 3, 3, 3]))
```

The IDE interface includes a top toolbar with icons for Run, Debug, Stop, Share, Save, and a dropdown menu for Language set to Python 3. Below the code editor is a console window with the following output:

```
None
5
3

...Program finished with exit code 0
Press ENTER to exit console.
```

7. Remove Duplicate Elements

CODE:

```
def unique(nums):
```

```
    result = []
```

```
    for x in nums:
```

```
        if x not in result:
```

```
            result.append(x)
```

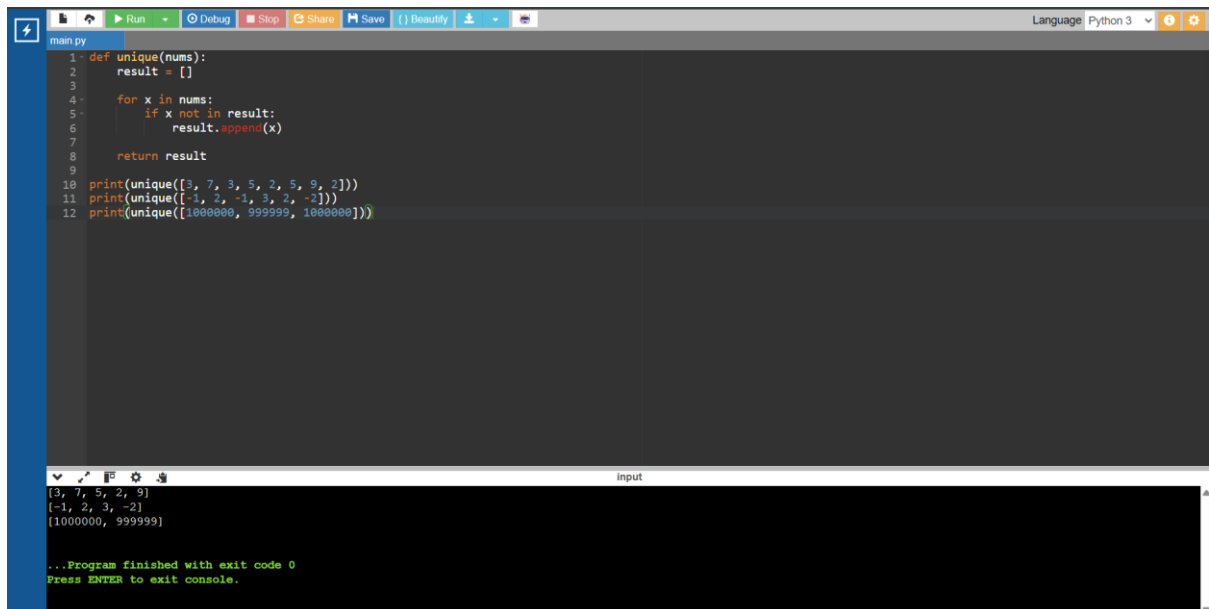
```
    return result
```

```
print(unique([3, 7, 3, 5, 2, 5, 9, 2]))
```

```
print(unique([-1, 2, -1, 3, 2, -2]))
```

```
print(unique([1000000, 999999, 1000000]))
```

OUTPUT:



The screenshot shows a Python IDE with a dark theme. The editor window displays the following code:

```
1 def unique(nums):
2     result = []
3
4     for x in nums:
5         if x not in result:
6             result.append(x)
7
8     return result
9
10 print(unique([3, 7, 3, 5, 2, 5, 9, 2]))
11 print(unique([-1, 2, -1, 3, 2, -2]))
12 print(unique([1000000, 999999, 1000000]))
```

The output console at the bottom shows the results of the three print statements:

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

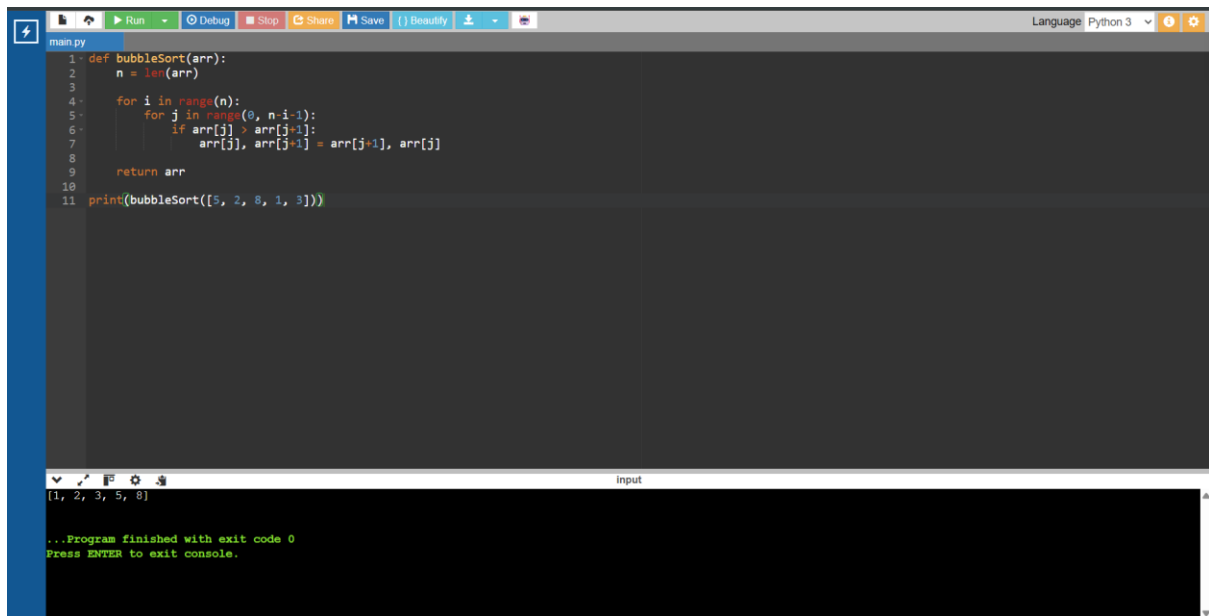
Below the output, the console indicates: "...Program finished with exit code 0" and "Press ENTER to exit console."

8. Bubble Sort

CODE:

```
def bubbleSort(arr):  
    n = len(arr)  
  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
  
    return arr  
  
print(bubbleSort([5, 2, 8, 1, 3]))
```

OUTPUT:



The screenshot shows a Python IDE with a dark theme. The editor window displays the bubble sort code. The top toolbar includes buttons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The console at the bottom shows the output of the program.

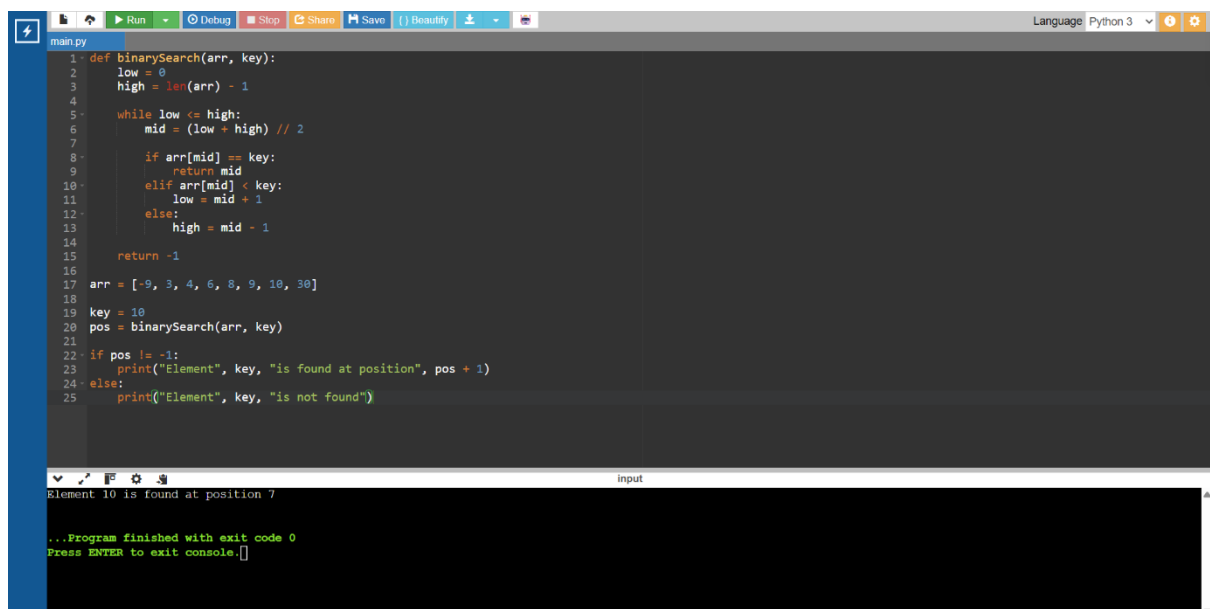
```
main.py  
1 def bubbleSort(arr):  
2     n = len(arr)  
3  
4     for i in range(n):  
5         for j in range(0, n-i-1):  
6             if arr[j] > arr[j+1]:  
7                 arr[j], arr[j+1] = arr[j+1], arr[j]  
8  
9     return arr  
10  
11 print(bubbleSort([5, 2, 8, 1, 3]))  
  
Input  
[1, 2, 3, 5, 8]  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

9. Binary Search

CODE:

```
def binarySearch(arr, key):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1  
  
arr = [-9, 3, 4, 6, 8, 9, 10, 30]  
key = 10  
pos = binarySearch(arr, key)  
  
if pos != -1:  
    print("Element", key, "is found at position", pos + 1)  
else:  
    print("Element", key, "is not found")
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py' with a dark theme. The code is as follows:

```
1 def binarySearch(arr, key):  
2     low = 0  
3     high = len(arr) - 1  
4  
5     while low <= high:  
6         mid = (low + high) // 2  
7  
8         if arr[mid] == key:  
9             return mid  
10        elif arr[mid] < key:  
11            low = mid + 1  
12        else:  
13            high = mid - 1  
14  
15    return -1  
16  
17 arr = [-9, 3, 4, 6, 8, 9, 10, 30]  
18  
19 key = 10  
20 pos = binarySearch(arr, key)  
21  
22 if pos != -1:  
23     print("Element", key, "is found at position", pos + 1)  
24 else:  
25     print("Element", key, "is not found")
```

The output console at the bottom shows the following text:

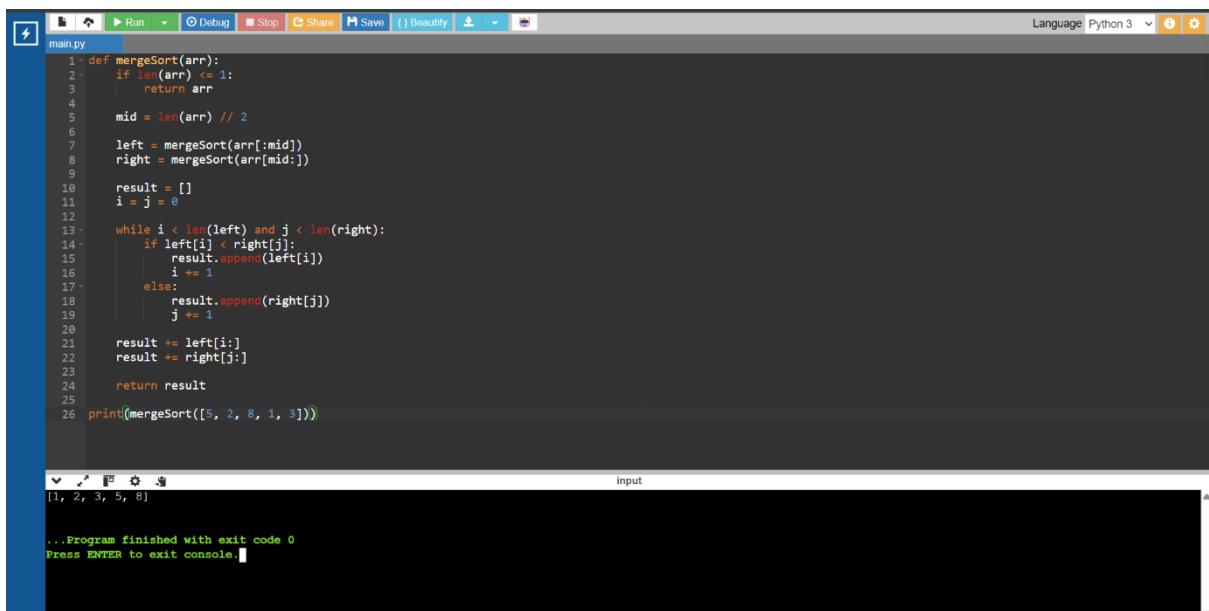
```
Element 10 is found at position 7  
...Program finished with exit code 0  
Press ENTER to exit console.
```

10. Sort Array in $O(n \log n)$

CODE:

```
def mergeSort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    mid = len(arr) // 2  
    left = mergeSort(arr[:mid])  
    right = mergeSort(arr[mid:])  
  
    result = []  
  
    i = j = 0  
  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
  
    result += left[i:]  
    result += right[j:]  
  
    return result  
  
print(mergeSort([5, 2, 8, 1, 3]))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py'. The code is the same as provided in the previous block. The output console at the bottom shows the result of the program execution: [1, 2, 3, 5, 8]. Below the output, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.'.

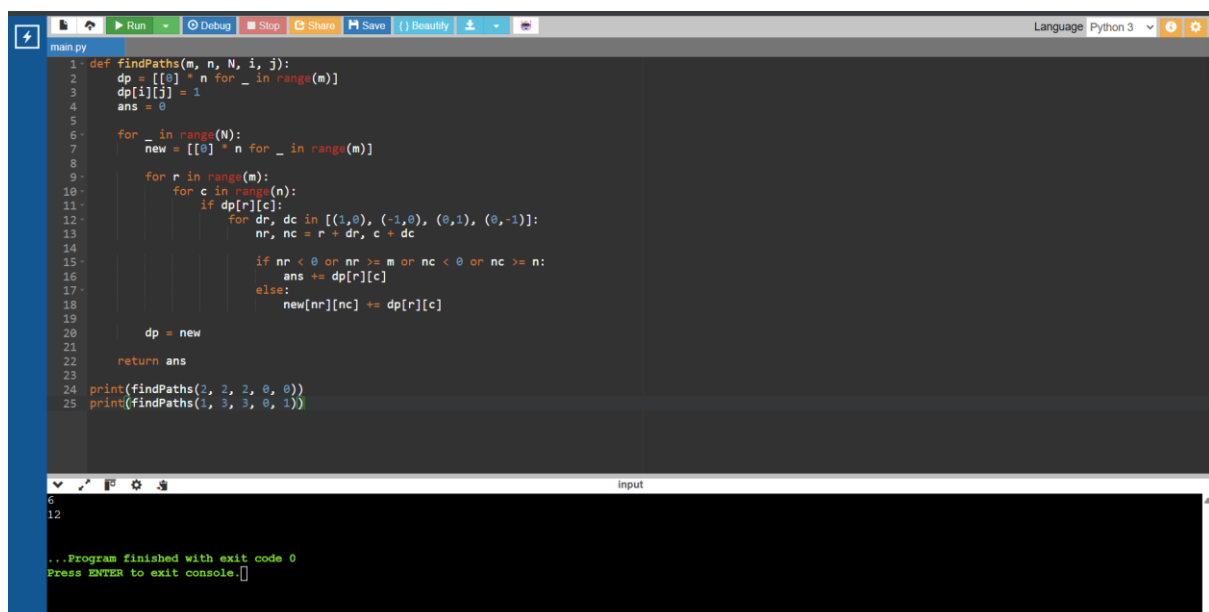
```
1 def mergeSort(arr):  
2     if len(arr) <= 1:  
3         return arr  
4  
5     mid = len(arr) // 2  
6     left = mergeSort(arr[:mid])  
7     right = mergeSort(arr[mid:])  
8  
9     result = []  
10  
11     i = j = 0  
12  
13     while i < len(left) and j < len(right):  
14         if left[i] < right[j]:  
15             result.append(left[i])  
16             i += 1  
17         else:  
18             result.append(right[j])  
19             j += 1  
20  
21     result += left[i:]  
22     result += right[j:]  
23  
24     return result  
25  
26 print(mergeSort([5, 2, 8, 1, 3]))  
  
[1, 2, 3, 5, 8]  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

11. Ball Out of Grid

CODE:

```
def findPaths(m, n, N, i, j):  
    dp = [[0] * n for _ in range(m)]  
    dp[i][j] = 1  
    ans = 0  
    for _ in range(N):  
        new = [[0] * n for _ in range(m)]  
        for r in range(m):  
            for c in range(n):  
                if dp[r][c]:  
                    for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:  
                        nr, nc = r + dr, c + dc  
                        if nr < 0 or nr >= m or nc < 0 or nc >= n:  
                            ans += dp[r][c]  
                        else:  
                            new[nr][nc] += dp[r][c]  
                else:  
                    new[nr][nc] += dp[r][c]  
        dp = new  
    return ans  
  
print(findPaths(2, 2, 2, 0, 0))  
print(findPaths(1, 3, 3, 0, 1))
```

OUTPUT:



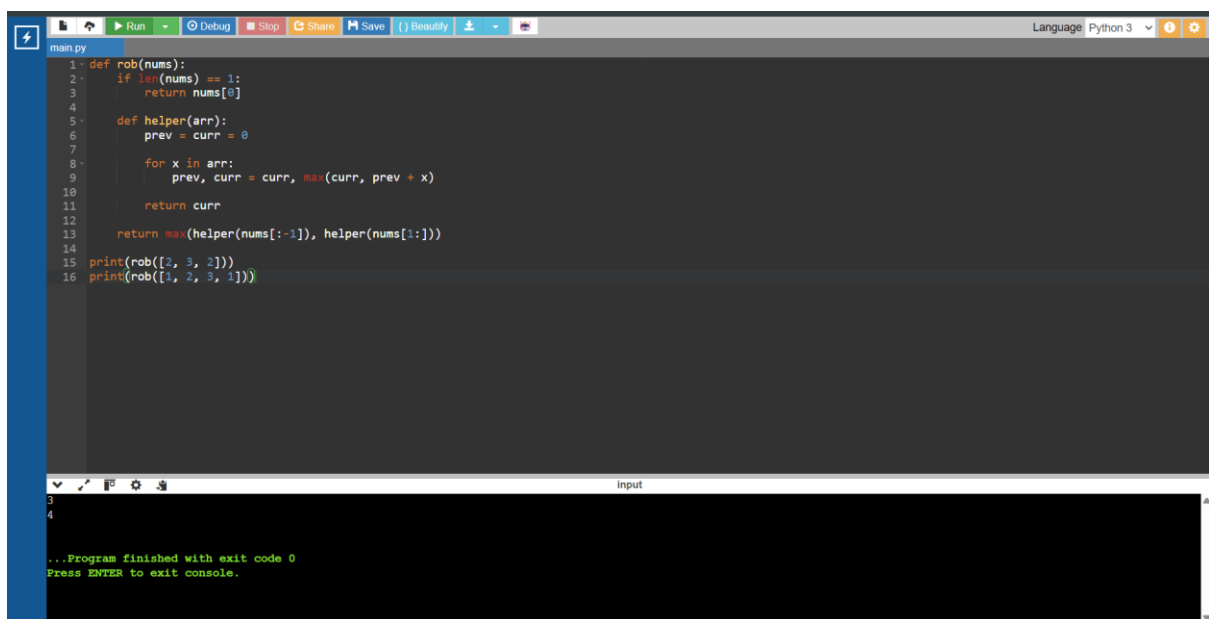
```
1 def findPaths(m, n, N, i, j):  
2     dp = [[0] * n for _ in range(m)]  
3     dp[i][j] = 1  
4     ans = 0  
5  
6     for _ in range(N):  
7         new = [[0] * n for _ in range(m)]  
8  
9         for r in range(m):  
10            for c in range(n):  
11                if dp[r][c]:  
12                    for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:  
13                        nr, nc = r + dr, c + dc  
14  
15                        if nr < 0 or nr >= m or nc < 0 or nc >= n:  
16                            ans += dp[r][c]  
17                        else:  
18                            new[nr][nc] += dp[r][c]  
19  
20                else:  
21                    new[nr][nc] += dp[r][c]  
22            dp = new  
23  
24    return ans  
25  
26 print(findPaths(2, 2, 2, 0, 0))  
27 print(findPaths(1, 3, 3, 0, 1))  
28  
29 ...Program finished with exit code 0  
30 Press ENTER to exit console.
```

12. House Robber II

CODE:

```
def rob(nums):  
    if len(nums) == 1:  
        return nums[0]  
  
    def helper(arr):  
        prev = curr = 0  
        for x in arr:  
            prev, curr = curr, max(curr, prev + x)  
        return curr  
  
    return max(helper(nums[:-1]), helper(nums[1:]))  
  
print(rob([2, 3, 2]))  
print(rob([1, 2, 3, 1]))
```

OUTPUT:

A screenshot of a Python IDE window. The top bar shows 'Language: Python 3'. The editor area contains the following code:

```
1 def rob(nums):  
2     if len(nums) == 1:  
3         return nums[0]  
4  
5     def helper(arr):  
6         prev = curr = 0  
7  
8         for x in arr:  
9             prev, curr = curr, max(curr, prev + x)  
10  
11         return curr  
12  
13     return max(helper(nums[:-1]), helper(nums[1:]))  
14  
15 print(rob([2, 3, 2]))  
16 print(rob([1, 2, 3, 1]))
```

The bottom panel shows the output:

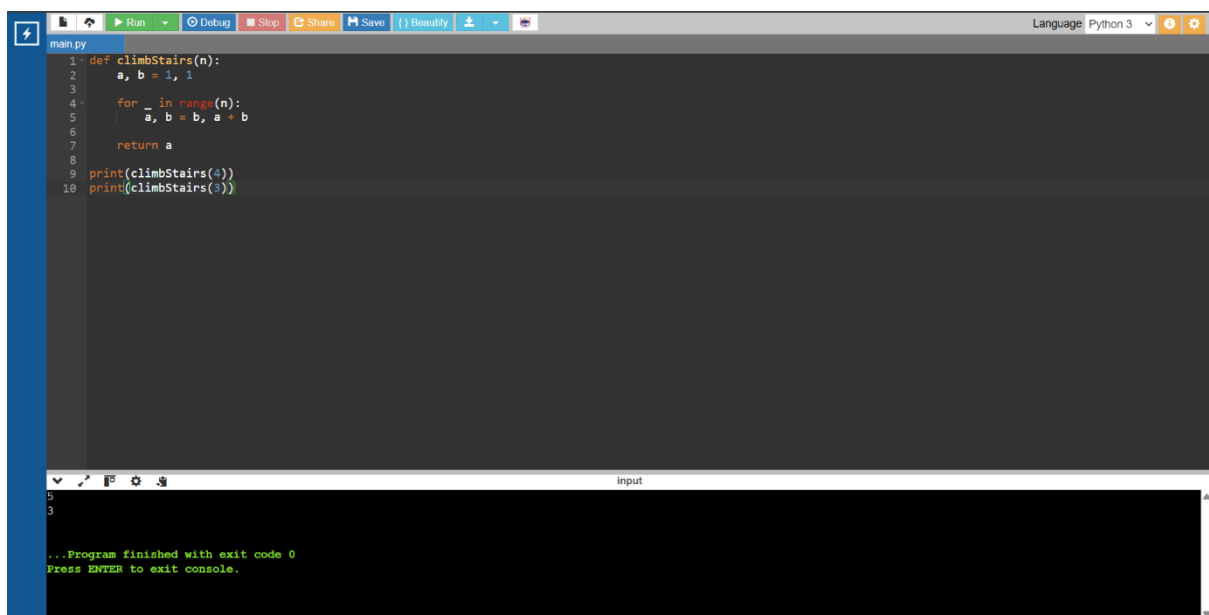
```
3  
4  
...Program finished with exit code 0  
Press ENTER to exit console.
```

13. Climbing Stairs

CODE:

```
def climbStairs(n):  
    a, b = 1, 1  
  
    for _ in range(n):  
        a, b = b, a + b  
  
    return a  
  
print(climbStairs(4))  
print(climbStairs(3))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py' with a toolbar at the top containing icons for Run, Debug, Stop, Share, Save, and Beautify. The code editor displays the following Python code:

```
1 def climbStairs(n):  
2     a, b = 1, 1  
3  
4     for _ in range(n):  
5         a, b = b, a + b  
6  
7     return a  
8  
9 print(climbStairs(4))  
10 print(climbStairs(3))
```

Below the code editor is a console window with the following output:

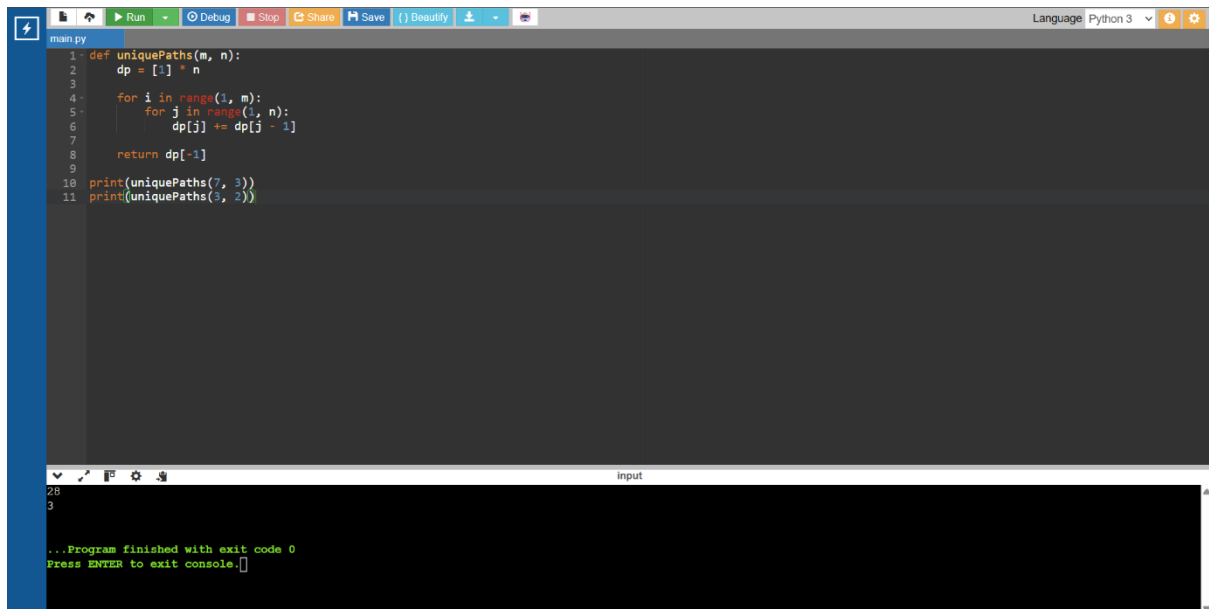
```
5  
3  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

14. Unique Paths

CODE:

```
def uniquePaths(m, n):  
    dp = [1] * n  
  
    for i in range(1, m):  
        for j in range(1, n):  
            dp[j] += dp[j - 1]  
  
    return dp[-1]  
  
print(uniquePaths(7, 3))  
print(uniquePaths(3, 2))
```

OUTPUT:

A screenshot of a Python IDE window titled 'main.py'. The code editor shows the same code as the previous block. The output console at the bottom shows the results of the program execution. The first line of output is '28', which is the result of uniquePaths(7, 3). The second line of output is '3', which is the result of uniquePaths(3, 2). Below these, the console shows the message '...Program finished with exit code 0' and 'Press ENTER to exit console.'.

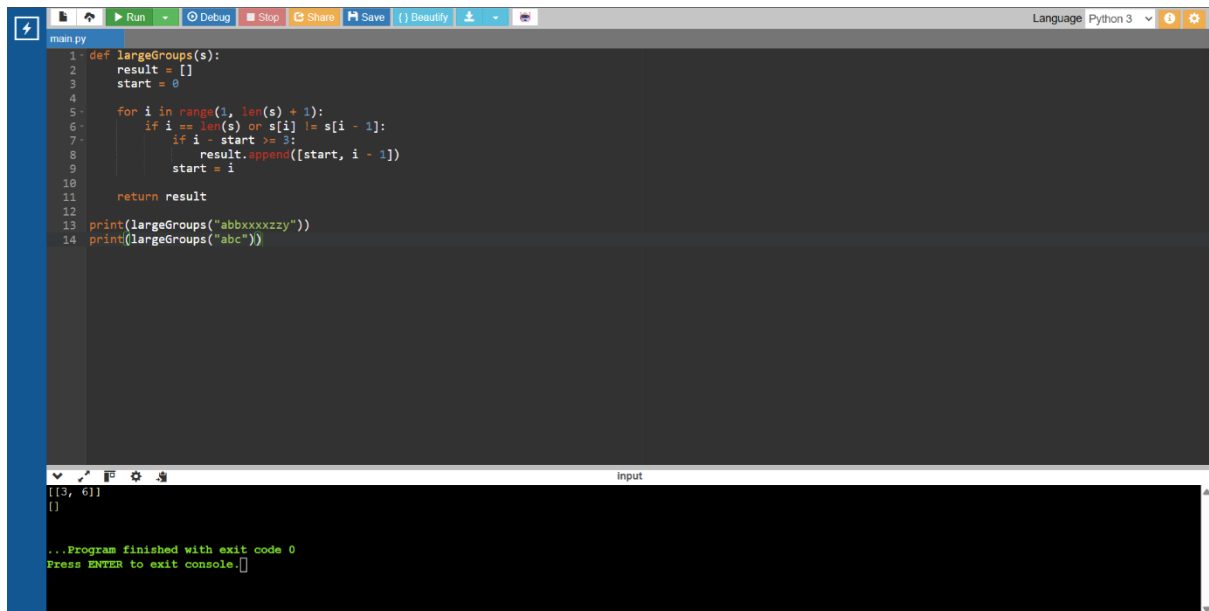
```
main.py  
1 def uniquePaths(m, n):  
2     dp = [1] * n  
3  
4     for i in range(1, m):  
5         for j in range(1, n):  
6             dp[j] += dp[j - 1]  
7  
8     return dp[-1]  
9  
10 print(uniquePaths(7, 3))  
11 print(uniquePaths(3, 2))  
  
28  
3  
...Program finished with exit code 0  
Press ENTER to exit console.
```

15. Large Groups in String

CODE:

```
def largeGroups(s):  
    result = []  
    start = 0  
    for i in range(1, len(s) + 1):  
        if i == len(s) or s[i] != s[i - 1]:  
            if i - start >= 3:  
                result.append([start, i - 1])  
            start = i  
    return result  
print(largeGroups("abbxxxxzzy"))  
print(largeGroups("abc"))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py' with the following code:

```
1 def largeGroups(s):  
2     result = []  
3     start = 0  
4  
5     for i in range(1, len(s) + 1):  
6         if i == len(s) or s[i] != s[i - 1]:  
7             if i - start >= 3:  
8                 result.append([start, i - 1])  
9             start = i  
10  
11     return result  
12  
13 print(largeGroups("abbxxxxzzy"))  
14 print(largeGroups("abc"))
```

The output console at the bottom shows the results of the program execution:

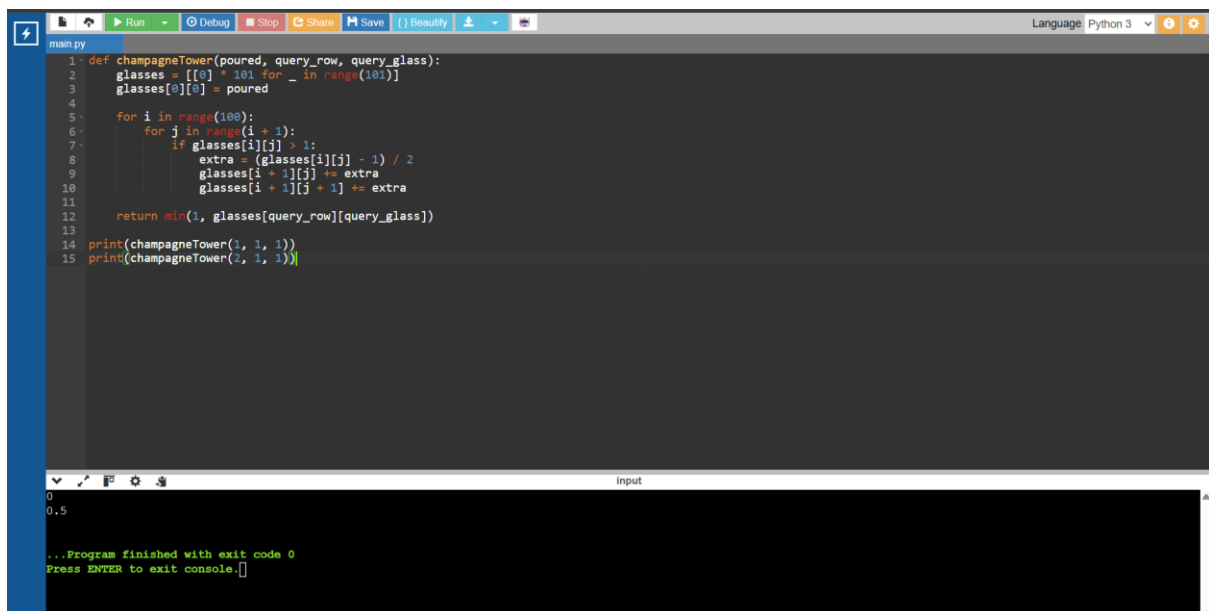
```
[[3, 6]]  
[]  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

16. Champagne Tower

CODE:

```
def champagneTower(poured, query_row, query_glass):  
    glasses = [[0] * 101 for _ in range(101)]  
    glasses[0][0] = poured  
    for i in range(100):  
        for j in range(i + 1):  
            if glasses[i][j] > 1:  
                extra = (glasses[i][j] - 1) / 2  
                glasses[i + 1][j] += extra  
                glasses[i + 1][j + 1] += extra  
    return min(1, glasses[query_row][query_glass])  
print(champagneTower(1, 1, 1))  
print(champagneTower(2, 1, 1))
```

OUTPUT:



The screenshot shows a Python IDE window titled 'main.py'. The code is the same as provided in the previous block. The output console at the bottom shows the results of the two print statements: 0 and 0.5. The console also displays the message '...Program finished with exit code 0' and 'Press ENTER to exit console.'

```
1 def champagneTower(poured, query_row, query_glass):  
2     glasses = [[0] * 101 for _ in range(101)]  
3     glasses[0][0] = poured  
4  
5     for i in range(100):  
6         for j in range(i + 1):  
7             if glasses[i][j] > 1:  
8                 extra = (glasses[i][j] - 1) / 2  
9                 glasses[i + 1][j] += extra  
10                glasses[i + 1][j + 1] += extra  
11  
12    return min(1, glasses[query_row][query_glass])  
13  
14 print(champagneTower(1, 1, 1))  
15 print(champagneTower(2, 1, 1))  
  
0  
0.5  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```